

Instruction of Python API for Leica AT 401

Shohta Takami

November 24, 2025

1 API structures

1.1 The Concept

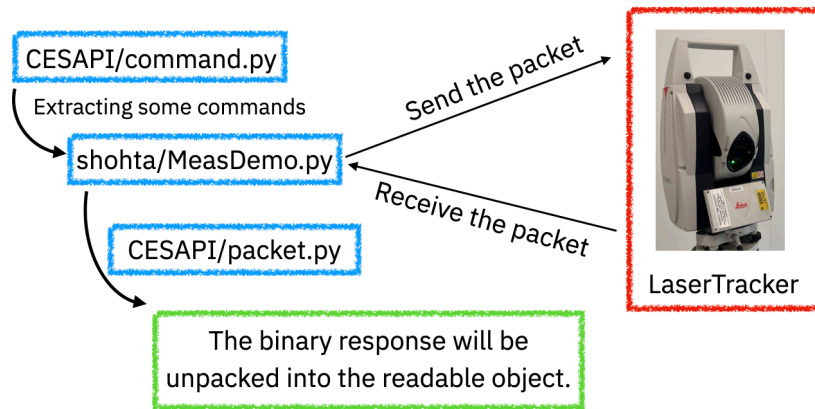


Figure 1: The general flow

The communication with Leica AT4xx will be conducted by binary packet protocol. You have to send and read the binary packet. The structure of the binary for each command is defined in the header file "Include/ES_C_API_Def.h". If you want to get the latest header file, you should make a contact with Hexagon customer support (<https://portable-mi.hexagon.com/>).

Figure.1 shows the conceptual flow of this API. "MeasDemo.py" is written by myself extracting commands from "command.py". "packet.py" provides functionality to convert numerically-defined command IDs into binary format, and to parse returned binary data into objects. command.py is responsible for sending the binaries generated by packet.py over TCP/IP and receiving binaries via TCP/IP. Then, the response from the laser tracker can be read by accessing the relevant instance variables of the returned object.

1.2 When you want to change the header file

"Include/ES_C_API_Def_clean.h" is the modified version of the header file to compatible for Python. When you want to use another header file, you have to regenerate "packet.py" and "command.py". The original header file "Includes/ES_C_API_Def.h" is written in C-style, which is not readable for Python. The file "ES_C_API_Def_clean.h" is modified to make it compatible for Python. Unnecessary macros, typedefs, and unused

structures have been removed, and overly nested definitions were flattened. Function and structure names were preserved, but formatting and alignment were adjusted to improve readability and automatic parsing by the Python generator script. It is assumed in the comments in the original header file that you will not have to switch the header file as long as you use AT4xx series.

2 The method to use commands

Script 1: The definition of the class for GetTrackerInfo

```

1 class GetTrackerInfoRT(object):
2     def __init__(self):
3         self.packet = b''
4         self.__packet_size = 164
5         self.__sizes = [148]
6         self.__formats = [('<I 64s i i i i d d d i i d d I i i ')]
7         self.packetInfo = BasicCommandRT()
8         self.packetInfo.packetHeader.lPacketSize = self.__packet_size
9         self.packetInfo.packetHeader.type = ES_DT_Command
10        self.packetInfo.command = ES_C_GetTrackerInfo
11        self.trackerType = int(0)
12        self.cTrackerName = b''
13        self.lSerialNumber = int(0)
14        self.lCompensationIdNumber = int(0)
15        self.bHasADM = int(0)
16        self.bHasOverviewCamera = int(0)
17        self.bHasNivel = int(0)
18        self.dNivelMountOffset = float(0)
19        self.dMaxDistance = float(0)
20        self.dMinDistance = float(0)
21        self.iMaxDataRate = int(0)
22        self.iNumberOfFaces = int(0)
23        self.dHzAngleRange = float(0)
24        self.dVtAngleRange = float(0)
25        self.accuracyModel = int(0)
26        self.iMajLCPFirmwareVersion = int(0)
27        self.iMinLCPFirmwareVersion = int(0)

```

Script 2: The structure of the binary for GetTrackerInfo defined in the header file

```

1 struct GetTrackerInfoRT
2 {
3     struct BasicCommandRT    packetInfo;
4     enum ES_LTSensorType     trackerType;
5     unsigned short          cTrackerName[32];
6     long                    lSerialNumber;
7     long                    lCompensationIdNumber;
8     ES_BOOL                 bHasADM;
9     ES_BOOL                 bHasOverviewCamera;
10    ES_BOOL                 bHasNivel;
11    double                   dNivelMountOffset;
12    double                   dMaxDistance;
13    double                   dMinDistance;
14    int                      iMaxDataRate;
15    int                      iNumberOfFaces;
16    double                   dHzAngleRange;

```

```

17     double                                dVtAngleRange;
18     enum ES_TrkAccuracyModel accuracyModel;
19     int                                     iMajLCPFirmwareVersion;
20     int                                     iMinLCPFirmwareVersion;
21 };

```

Here, I explain how to use "GetTrackerInfo" command as an example, which is for retrieving the basic information of the laser tracker. First, you have to make an instance of all commands from "CESAPI/command.py". Each function in "CESAPI/command.py" also includes the feature to send the packet by the function "execute". So, you will get the object corresponding to the function as returned value, after executing the function. The structure of each returned object is defined in the class namely suffixed with "RT" to the name of the command you used (shown as Script.1). You can also look up to "Includes/ES_C_API_Def.h", which "CESAPI/command.py" is originated from. Here is the example flow of access to returned object.

Script 3: The flow to get the tracker's name

```

1  command = CommandSync() #make an instance of command
2  tkrinfo = command.GetTrackerInfo()
3  tkrname = tkrinfo.cTrackerName
4  print(f"{tkrname}")
5  There is one exception for this naming convention.
6  When you measure the coordinate, you. have to use

```

2.1 The command to take coordinates

Regarding of the command to take coordinates, the naming convention explained above will be different. When you measure the coordinate of the pointing reflector, you would have to use command named "StartMeasurement". For this command, the returned object will differ depending on the status. If the tracker returns some error, the returned object will be "StartMeasurementRT". When the command works successfully, the object named "SingleMeasResultT" will be returned (shown in Script.4). So you should look up to the structure of "SingleMeasResultT" when you use "StartMeasurement" to measure the coordinate.

Script 4: SingleMeasResultT is returned as a response to "StartMeasurement"

```

1  struct SingleMeasResultT
2  {
3      struct ReturnDataT packetInfo;
4      enum ES_MeasMode    measMode;
5      ES_BOOL             bIsTryMode;
6      double              dVal1;
7      double              dVal2;
8      double              dVal3;
9      double              dStd1;
10     double              dStd2;
11     double              dStd3;
12     double              dStdTotal;
13     double              dPointingError1;
14     double              dPointingError2;
15     double              dPointingError3;
16     double              dAprioriStd1;
17     double              dAprioriStd2;

```

```

18     double          dAprioriStd3;
19     double          dAprioriStdTotal;
20     double          dTemperature;
21     double          dPressure;
22     double          dHumidity;
23 };

```

dVal1 to dVal3 corresponds to the coordinates. The numbering follows the same ordering of coordinates as described in the documentation (TPI3.0_ProgrammersManual.pdf). Basically, I applied the spherical counter-clockwise system in Figure.2 following EMMA. In this case, dVal1 is H, dVal2 is V, dVal3 is D.

- ES_CS_SCC
Spherical Counter-Clockwise system.

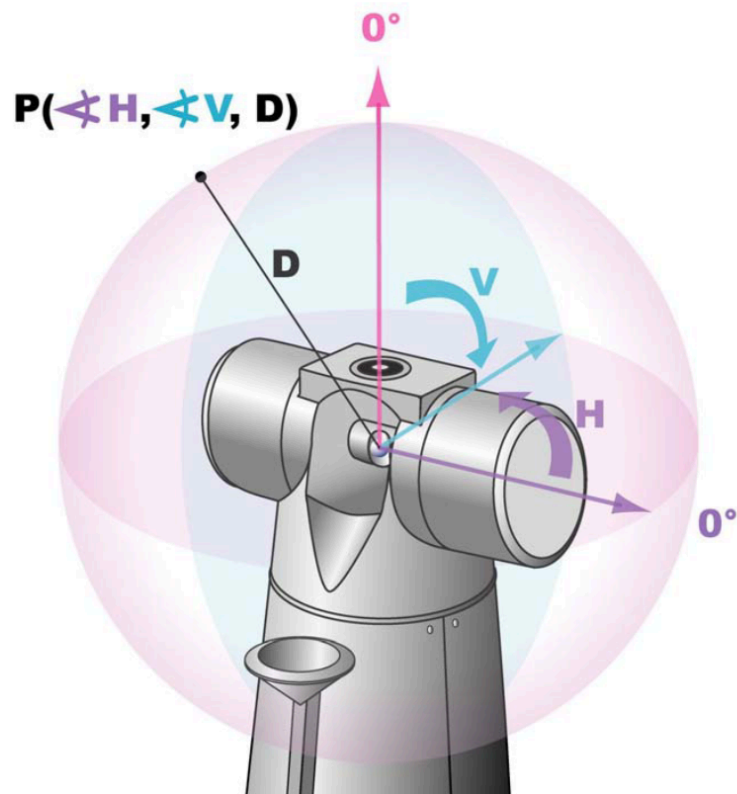


Figure 2: Spherical Counter-Clockwise System

3 Changing configurations

Table 1: The configuration list

1. Measurement mode
2. Weather monitor
3. Orientation Params
4. Transformation Params
5. Units
6. Coordinate system type

Table.1 shows the lists of configurations. You have to set the configuration actively before measurement. In this section, I explain how to modify the settings by using the unit-system configuration as an example.

Script 5: The scripts to change configuration

```
1 def set_units():
2     units = SystemUnitsDataT()
3     units.lenUnitType = ES_LU_Millimeter
4     units.angUnitType = ES_AU_Radian
5     units.tempUnitType = ES_TU_Fahrenheit
6     units.pressUnitType = ES_PU_InHg
7     units.humUnitType = ES_HU_RH
8     command.SetUnits(units)
9     print(f'Units set successfully')
```

Script.5, extracting from "shohta/MeasDemo.py" shows how to change the configuration of units system. The way to change configuration is all the same. First, you have to make an instance of configuration (pick right one, easily estimated from its name). Second, you just have to define each instance variables to configuration as you like. The definition of units are defined in "Include/Enum.h" and those of other configurations are defined in "Include/ES_C_API_Def.h". You can also find the definition of these configuration in the first few hundreds of lines in "CESAPI/packet.py". After creating the instance of your configuration as above, you can apply the configuration by the "Setxxx" function.

4 Practical use of MeasDemo.py

"shohta/MeasDemo.py" is the script I actually used. It imports "CESAPI/connection.py" and "CESAPI/command.py", "CESAPI/packet.py". I organized the functions according to the calling sequence. The main function is for the repeated measurement explained below.

4.1 TCP/IP Connection

First, you have to connect to the laser tracker. The default address is "192.168.0.2", and the port number is 700. You have to make an instance of the class in CESAPI/

`connection.py`. Then, connect to the laser tracker using the function in this class. After successfully connection, you have to input the instance of connection to the object `CommandSync` in `"CESAPI/comand.py"`.

Script 6: The command to make a connection

```
1 conn = Connection()
2 conn.connect(host='192.168.0.2', port=700)
3 ommand = CommandSync(conn)
```

4.2 Before measurement

You have to set the configurations in Table.1 as I explained above before the measurement. At that moment, I followed the EMMA's configuration. We may have to figure out the appropriate configuration.

You have to initialize before measurement (The second command in Table.2). Lastly, you have to execute `"set_system_setting"` to apply every configuration so far. "1" means "Apply", and "0" means "Not Apply".

Table 2: Commands to set configurations

1. `set_units()`
2. `initialization`
3. `set_weathermonitor()`
4. `set_TransformationParam()`
5. `set_OrientationpParam()`
6. `set_measurement_mode()`
7. `set_CoordinateSystemType(7)`
8. `set_system_setting()`

After all these setting, you will be ready to operate the tracker.

4.3 The operation of the laser tracker

The operations I needed were just "pointing the laser to arbitrary coordinate" and "measuring the coordinates".

The function, `"go_to_position"`, sends the command to point the laser to the arbitrary point. The values of 3 coordinates as you configured is required as arguments. According to `"TPI3.0_ProgrammersManual.pdf"`, you have to input `"True"` to the fourth argument `"bUseADM"` because AT4xx series has ADM (Absolute Distance Meter). The tracker will automatically do the spiral search if it can't find any reflector at the input coordinate.

The function, `"point_measure"`, sends the command to measure the coordinate and repeat the measurement input times. In this function, the measured data and measurement information are recorded to the csv file and root file which contains TTree object as a log. Table.3 shows columns of this log file.

Table 3: The list of the log file

1. coordinate system type
2. point name
3. measurement id
4. measurement mode
5. date
6. time
7. coordinates
8. standard deviations of each coordinate
9. the total standard deviation

The coordinate system type will be recorded by the number defined in the header file or `"packet.py"`. The measurement id specifies the order of the measurement in a series of repeated measurements. (I regret not adding the environmental information such as the temperature even though you can easily access them as the measured coordinates.) I tried 100-time repeated measurement and it took roughly 30 minutes. One unexpected error is that measurements sometimes start to fail. I could not identify why this happens, but the configuration for the weather monitor somehow was changed to `"ReadOnly"` every time; nevertheless it should be `"ReadandCalculateRefractions"`.